

CO4 – AT01: Security Testing Simulation

Answer-Preparation Study Outline (Not a Final Submission)

Course: Ethical Hacking (ITA1410) | Course Outcome: CO4 | Total Marks: 100

This outline is a study aid, not a completed answer sheet. All testing must be performed only on the specific target and within the scope your faculty has approved — never on public or third-party systems. Build your own report from your own evidence, then complete the Code of Conduct declaration yourself.

Question 1: Web Server Reconnaissance and Vulnerability Analysis

Gather OSINT (Google/WHOIS), run basic network recon (Ping/Tracert/IPConfig/Netstat), Nmap port/service scan, WhatWeb tech fingerprinting, HTTP header analysis (Curl/Wget), Nikto CGI/web-server scan, and Gobuster/Dirb content discovery. Classify findings by severity and recommend hardening.

Concepts you need to understand

- Passive vs. active reconnaissance — OSINT (Google, WHOIS) is passive; Nmap/Nikto/Gobuster are active.
- What WHOIS reveals about domain ownership/infrastructure, and its limits for internal lab targets.
- HTTP response headers can leak server software, frameworks, and misconfigurations (e.g., missing security headers, verbose error pages).
- Directory/file brute-forcing finds content not linked from the site (backups, admin panels, config files).

Tools & example command syntax (reference only — use only against your faculty-approved lab target, in scope)

```
whois <target-domain>          # if applicable to your assigned target
ping <target> / tracert <target> / ipconfig /all / netstat -ano
nmap -sV -sC -O <target>
whatweb <target>
curl -I http://<target>
nikto -h http://<target>
gobuster dir -u http://<target> -w <wordlist>
```

What your report should cover

- Organize findings by recon phase: OSINT → network recon → port/service scan → web tech fingerprint → header analysis → Nikto → content discovery.
- For each discovered admin page/backup/config file, explain what it exposes and the impact if found by an attacker.
- Classify all findings into severity tiers (Critical/High/Medium/Low/Info) with justification.
- Recommend concrete hardening: remove exposed backups, disable directory listing, add missing security headers, patch outdated components.

Mapping to the rubric

Tool Selection & Configuration (20):

Justify why each tool was chosen for the step and show it was configured correctly.

Testing Procedure & Execution (20):

Document a systematic, in-scope, well-documented sequence of steps — never anything destructive or unauthorized.

Identification & Validation of Vulnerabilities (20):

Don't just paste tool output — manually confirm findings and separate real issues from false positives.

Risk Analysis & Remediation (20):

For every confirmed finding: cause, likelihood, impact, severity, and a practical fix.

Evidence, Report Quality & Ethical Compliance (20):

Commands, screenshots, references, professional formatting, and explicit statement that testing stayed within authorized scope.

Question 2: Web Application Vulnerability Assessment

Identify the app's technologies/attack surface, examine inputs/cookies/sessions/requests, and test for SQL Injection, XSS, insecure authentication, session-management weaknesses, directory traversal, and misconfigurations using Burp Suite/OWASP ZAP/Nikto/Nmap/Curl/browser devtools.

Concepts you need to understand

- OWASP Top 10 categories relevant here: Injection, Broken Authentication, Sensitive Data Exposure, Security Misconfiguration.
- How a proxy tool (Burp/ZAP) intercepts and lets you modify requests to test input handling.
- Difference between reflected, stored, and DOM-based XSS.
- What makes session management 'insecure' (predictable tokens, missing HttpOnly/Secure flags, no session expiry).

Tools & example command syntax (reference only — use only against your faculty-approved lab target, in scope)

```
# Configure browser to proxy through Burp Suite / OWASP ZAP (e.g., 127.0.0.1:8080)
nmap -sV <target>
nikto -h http://<target>
curl -i -X POST http://<target>/login -d "user=test&pass=test'"
# Use ZAP's Active Scan / Burp's Scanner (where licensed) for automated coverage, then
validate manually
```

What your report should cover

- Document each input field/parameter tested and the technique applied (e.g., SQLi payload attempted, XSS payload attempted).
- For each confirmed vulnerability: evidence (request/response), impact, and how it could be exploited further.
- Classify vulnerabilities by severity using a consistent scale (e.g., CVSS-style High/Medium/Low).
- Recommend remediation and secure coding practices: parameterized queries, output encoding, secure cookie flags, session regeneration on login.

Mapping to the rubric

Tool Selection & Configuration (20):

Justify why each tool was chosen for the step and show it was configured correctly.

Testing Procedure & Execution (20):

Document a systematic, in-scope, well-documented sequence of steps — never anything destructive or unauthorized.

Identification & Validation of Vulnerabilities (20):

Don't just paste tool output — manually confirm findings and separate real issues from false positives.

Risk Analysis & Remediation (20):

For every confirmed finding: cause, likelihood, impact, severity, and a practical fix.

Evidence, Report Quality & Ethical Compliance (20):

Commands, screenshots, references, professional formatting, and explicit statement that testing stayed within authorized scope.

Question 3: Security Header and HTTPS Configuration Assessment

Use Curl for HTTP response headers and Nmap SSL scripts for TLS analysis. Identify missing/misconfigured security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options), verify HTTP→HTTPS redirection, and assess TLS protocol/cipher/certificate issues.

Concepts you need to understand

- Purpose of each header: CSP (limits allowed content sources), HSTS (forces HTTPS), X-Frame-Options (clickjacking defense), X-Content-Type-Options (MIME sniffing defense).
- Weak vs. strong TLS versions (TLS 1.0/1.1 deprecated; TLS 1.2/1.3 preferred) and weak cipher suites (e.g., RC4, export ciphers).
- What a certificate issue looks like: expired, self-signed, wrong hostname, weak key length.

Tools & example command syntax (reference only — use only against your faculty-approved lab target, in scope)

```
curl -I https://<target>
curl -I http://<target>          # check for redirect to HTTPS
nmap --script ssl-enum-ciphers -p 443 <target>
nmap --script ssl-cert -p 443 <target>
```

What your report should cover

- List every header checked with present/absent/misconfigured status.
- State whether HTTP→HTTPS redirection works and any gaps.
- List TLS protocol versions and cipher suites supported, flagging weak ones.
- Report certificate details (issuer, validity dates, common name) and any issues found.
- Classify severity per finding and recommend specific header values/TLS config changes.

Mapping to the rubric

Tool Selection & Configuration (20):

Justify why each tool was chosen for the step and show it was configured correctly.

Testing Procedure & Execution (20):

Document a systematic, in-scope, well-documented sequence of steps — never anything destructive or unauthorized.

Identification & Validation of Vulnerabilities (20):

Don't just paste tool output — manually confirm findings and separate real issues from false positives.

Risk Analysis & Remediation (20):

For every confirmed finding: cause, likelihood, impact, severity, and a practical fix.

Evidence, Report Quality & Ethical Compliance (20):

Commands, screenshots, references, professional formatting, and explicit statement that testing stayed within authorized scope.

Question 4: Broken Access-Control Assessment

Using OWASP ZAP, capture requests for student-record access, identify object identifiers, and — using only faculty-provided test identifiers — determine if one student can access another's records or a normal user can reach admin-only resources. Compare authorized vs. unauthorized responses.

Concepts you need to understand

- IDOR (Insecure Direct Object Reference) — accessing another user's data by changing an ID parameter.
- Authentication failure (who you are wasn't verified) vs. authorization failure (you were verified but shouldn't have access to this resource).
- Why this test must stay strictly within faculty-provided test accounts/identifiers — never live or third-party data.

Tools & example command syntax (reference only — use only against your faculty-approved lab target, in scope)

```
# Log in as Student A in browser proxied through OWASP ZAP; capture the record-view request
# Note the object identifier (e.g., ?studentId=1042) in the captured request
# Replay the same request in ZAP's manual request editor, substituting a DIFFERENT faculty-provided test ID
# Repeat the same approach for a normal-user request against an admin-only endpoint
```

What your report should cover

- Show the captured request/response for the authorized case and the substituted-ID (test) case side by side.
- State clearly whether unauthorized access was possible, and to what data/resource.
- Explain whether this is an authentication or authorization failure and why.
- Classify severity based on data sensitivity and ease of exploitation.
- Recommend fixes: server-side ownership checks, role-based access control, indirect object references instead of raw sequential IDs.

Mapping to the rubric

Tool Selection & Configuration (20):

Justify why each tool was chosen for the step and show it was configured correctly.

Testing Procedure & Execution (20):

Document a systematic, in-scope, well-documented sequence of steps — never anything destructive or unauthorized.

Identification & Validation of Vulnerabilities (20):

Don't just paste tool output — manually confirm findings and separate real issues from false positives.

Risk Analysis & Remediation (20):

For every confirmed finding: cause, likelihood, impact, severity, and a practical fix.

Evidence, Report Quality & Ethical Compliance (20):

Commands, screenshots, references, professional formatting, and explicit statement that testing stayed within authorized scope.

Question 5: Automated Scanning and Manual Validation

Use Nikto for server misconfigurations and Wapiti or OWASP ZAP for automated app scanning. Compare tool outputs, categorize into misconfiguration/information-disclosure/authentication/injection issues, manually validate at least three findings, and assign severity.

Concepts you need to understand

- Automated scanners produce false positives — manual validation is what separates a real report from a raw tool dump.
- How to categorize a finding: does it stem from server config, from data leakage, from weak auth, or from unsanitized input?
- What counts as 'requires further investigation' vs. 'confirmed' vs. 'false positive'.

Tools & example command syntax (reference only — use only against your faculty-approved lab target, in scope)

```
nikto -h http://<target>
wapiti -u http://<target>          # or run an OWASP ZAP Active Scan
# For each selected finding, manually reproduce it (e.g., resend the flagged request with
curl/Burp) to confirm
```

What your report should cover

- Build a comparison table: Finding | Reported by (Nikto/Wapiti/ZAP) | Category | Status (Confirmed/False Positive/Needs Investigation).
- For the 3+ manually validated findings, document exactly how you validated each one and what you observed.
- Assign severity based on likelihood and impact for confirmed findings only.
- Give a remediation recommendation per confirmed vulnerability, plus an overall conclusion on the app's security posture.

Mapping to the rubric

Tool Selection & Configuration (20):

Justify why each tool was chosen for the step and show it was configured correctly.

Testing Procedure & Execution (20):

Document a systematic, in-scope, well-documented sequence of steps — never anything destructive or unauthorized.

Identification & Validation of Vulnerabilities (20):

Don't just paste tool output — manually confirm findings and separate real issues from false positives.

Risk Analysis & Remediation (20):

For every confirmed finding: cause, likelihood, impact, severity, and a practical fix.

Evidence, Report Quality & Ethical Compliance (20):

Commands, screenshots, references, professional formatting, and explicit statement that testing stayed within authorized scope.

General Write-Up Checklist Before Submission

- Explicit statement at the top of your report confirming testing stayed within the faculty-approved scope.
- Real screenshots/output for every tool step — never fabricated results.
- A consolidated severity-classified findings table across all five questions.
- Clear separation between 'what the tool reported' and 'what you personally validated'.
- Your name/reg. no. filled into the Code of Conduct and signed.